1 TRIMESTRE: DESAFIO PSP Y DIN.

Indicaciones

- El uso de herramientas de Inteligencia Artificial (IA), como ChatGPT, Copilot u otras similares, **no está permitido para generar total o parcialmente el trabajo que debe realizar el alumno**.
- Estas herramientas podrán utilizarse únicamente como apoyo al aprendizaje, por ejemplo, para resolver dudas, comprender conceptos o aprender a desarrollar una tarea. Su uso deberá ser siempre responsable y transparente.
- Durante la evaluación, se podrán realizar preguntas (tanto escritas, como orales) sobre el trabajo entregado para comprobar que el alumno comprende su contenido y es capaz de explicar las decisiones tomadas y el proceso seguido.
- **Si se detecta un uso indebido de la IA, o si el alumno no demuestra comprender el trabajo presentado, la tarea se calificará como suspenso.**

Se deben enviar cuatro partes:

1. Código de las aplicaciones. **El código debe estar documentado por uno mismo.**
2. Archivos SQL para crear las BDs y poder meter datos de inicio en las BDs (CREATE TABLE e INSERT INTO)
3. Archivos JSON (archivo enviados y recibidos por ambas aplicaciones y el archivo JSON que lee/escribe la segunda aplicación para enseñar el registro de peticiones)
4. Elaborar un documento que explique de forma clara y ordenada las distintas partes del proyecto y el proceso seguido para desarrollarlo. El documento deberá incluir imágenes, capturas de pantalla, esquemas u otros recursos visuales que ayuden a comprender el trabajo realizado.
    - **Descripción del proyecto:** explicar brevemente en qué consiste, cuál es su objetivo y qué se pretende conseguir.
    - **Partes del proyecto:** identificar y explicar los principales componentes que lo forman.
    - **Proceso de desarrollo:** describir, paso a paso y en orden cronológico, las tareas realizadas desde el inicio hasta la finalización del proyecto, indicando quién se ha encargado de realizar cada parte.
    - **Imágenes:** incluir capturas de pantalla, fotografías o esquemas que muestren las diferentes fases del trabajo. Cada imagen deberá ir acompañada de una breve explicación de lo que representa.
    - **Dificultades y soluciones:** señalar los problemas encontrados durante el desarrollo y explicar cómo se resolvieron.

- En la defensa del proyecto se podrán hacer preguntas directas sobre el código y la aplicación (CUALQUIER PARTE DEL CÓDIGO) Se mirará la app **ejecutándose en tiempo real**, en el ordenador del alumno.
- Se podrán realizar preguntas escritas sobre el trabajo entregado para comprobar que el alumno comprende su contenido y es capaz de explicar las decisiones tomadas y el proceso seguido.
- Se debe subir los cambios realizados en GITHUB, la carpeta creada para el proyecto debe ser pública. Compartir el enlace con el profesor (Subirlo en La Salle - Sellenet)

Objetivo del trabajo

El proyecto integra los contenidos de las asignaturas de Diseño de Interfaces y Programación de Servicios y Procesos (PSP).

Desarrollar, en parejas, un sistema de gestión compuesto por dos aplicaciones de escritorio independientes en C#, donde la primera aplicación implementa operaciones CRUD sobre una base de datos relacional y se comunica con la segunda mediante Named Pipes, la segunda aplicación se encargará de procesar los datos.

→Aplicación 1 — Sistema de gestión (WPF)

1. La primera aplicación permitirá gestionar información almacenada en una base de datos relacional mediante operaciones CRUD.
2. Además, enviará solicitudes a la segunda aplicación serializados en formato JSON mediante named Pipelines y recibirá los datos serializados en formato JSON.

→Aplicación 2 — Servicio de procesamiento

1. Será un proceso independiente que recibirá mensajes mediante Named Pipes, deserializará los datos, ejecutará una operación de procesamiento y devolverá una respuesta serializada a la primera aplicación.
2. La segunda aplicación tiene a su vez tiene una interfaz gráfica en WPF (No es un CRUD), la cual trabaja sobre archivos JSON e HILOS - DISPATCHER.

→Aplicación 1 — Sistema de gestión (WPF)

Será la aplicación principal del sistema. Simulará un sistema de gestión relacionado con un ámbito a elegir, como una escuela, un hotel, una biblioteca, una clínica, una empresa u otro escenario previamente aprobado.

Permitirá gestionar la información mediante una interfaz gráfica WPF, validar los datos introducidos y comunicarse con la segunda aplicación para solicitar operaciones de procesamiento.

La aplicación utilizará una base de datos relacional y enviará las solicitudes mediante Named Pipes, serializando los datos en formato JSON. Recibirá la respuesta de la segunda aplicación y, cuando corresponda, guardará los resultados en la base de datos para poder consultarlos posteriormente.

1 Requisitos de la base de datos

La base de datos deberá cumplir:

1. Incluir **un mínimo de cuatro tablas relacionadas**.
2. Los datos recibidos por la segunda aplicación se deben guardar en la base de datos.
3. Implementar, como mínimo, una relación **uno a varios (1)** y una relación **varios a varios (N)**.
4. Incluir una tabla intermedia para la relación varios a varios que contenga **al menos un atributo propio de la relación**, además de las claves que la vinculan con las otras tablas.
5. Utilizar claves primarias y claves foráneas correctamente.
6. Definir las acciones referenciales ON DELETE y ON UPDATE adecuadas para mantener la integridad de los datos. Se deberán utilizar las opciones CASCADE y/o SET NULL cuando proceda y sean compatibles con las reglas del sistema.
7. Incorporar datos coherentes y suficientes para probar las operaciones CRUD y el procesamiento de solicitudes.

2 Arquitectura de la aplicación

1. El proyecto deberá seguir los patrones:

- **MVVM (** Model → entidades, ViewModel → lógica y comandos, View → interfaz WPF)

2. Crear Datos - Validar datos antes de guardar (Por validación de datos, se refiere por ejemplo que un DNI no pueda repetirse, una edad sea mayor que cero, un salario negativo…)
3. Modificar Datos - Validar datos antes de realizar los cambios (ten en cuenta la referencia ON UPDATE para tablas relacionadas)
4. Cargar Datos
5. Borrar Datos (ten en cuenta la referencia ON DELETE para tablas relacionadas)
6. Gestionar selección del DataGrid
7. La aplicación deberá hacer uso de **Data Binding**, evitando el acceso directo a controles desde el código. Implementación de INotifyPropertyChanged

3 Interfaz de Usuario

No se evaluará solo que funcione, sino cómo se presenta y se usa.

Requisitos generales de la interfaz

- Uso correcto de márgenes, alineaciones y tamaños
- Textos legibles y bien etiquetados

3.1 Enlace de datos (UI + ViewModel)

La interfaz debe comunicarse con la lógica mediante **Data Binding**, no mediante acceso directo a controles.

Requisitos:

- Enlaces bidireccionales (TwoWay)
- Uso de **SelectedItem**
- **Actualización automática de la interfaz**

3.2 Controles WPF OBLIGATORIOS

La aplicación debe usar los siguientes controles:

Controles de entrada de datos

- TextBox → para nombres, descripciones, emails, etc.
- DatePicker → para fechas
- ComboBox → para claves foráneas
- CheckBox
- Button
- ListBox
- PasswordBox

Visualización de datos

- DataGrid

Organización visual

- Grid
- StackPanel , DockPanel
- GroupBox - Border para separar zonas visuales

Controles comunes

Label, Image, Menu…

→Aplicación 2 — Servicio de procesamiento

La segunda aplicación será una aplicación WPF (NO TIENE BDs, recomiendo utilizar un DataGrid enlazado a una colección ObservableCollection<PeticicionRegistro>) independiente que actuará como procesador de peticiones y herramienta de supervisión. Recibirá datos de la aplicación principal, los deserializará, comprobará su validez, ejecutará una operación, registrará lo sucedido en su interfaz y devolverá una respuesta.

Debe cumplir las siguientes partes:

1. La comunicación entre dos procesos independientes mediante Named Pipes (PARTE 1).
    - Debe recibir los datos serializados
    - Desrealiza, valida y **procesa los datos** → Se debe hacer internamente algunos cálculos dentro de la aplicación.
    - Registro de peticiones → Publica los eventos que la interfaz debe mostrar. Muestra peticiones, estados y errores.
    - Envía los datos a la aplicación principal serializados.
2. La coordinación entre el código que recibe las peticiones en segundo plano y la interfaz gráfica de WPF. → Usar DISPATCHER (PARTE 2) Obligatorio el uso de HILOS

```
await Dispatcher.InvokeAsync(() =>
{
   dgPeticiones.Items.Add("Petición recibida");
});
```

3. Los se guardan y se leen desde un archivo JSON, al abrir la app debe cargar dichos datos.

→ PARTES DE LA INTERFAZ

1 Registro de peticiones

Será el historial de las solicitudes recibidas desde la aplicación principal. Al seleccionar una fila, se podrá abrir el detalle de esa petición. Recomiendo utilizar un DataGrid enlazado a una colección ObservableCollection<PeticicionRegistro> o a un modelo equivalente.

Columnas:

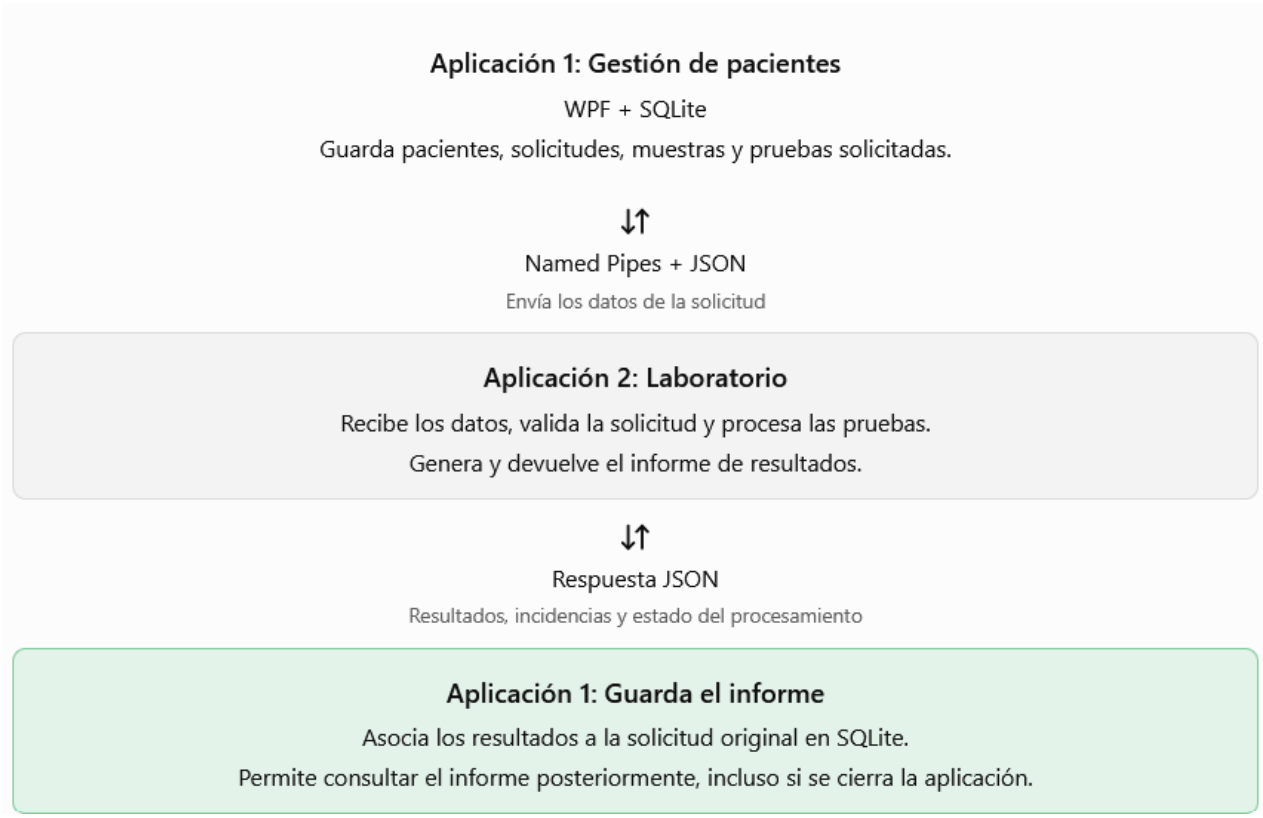
- ID de petición (Enviado por la aplicación principal).
- Fecha y hora de recepción.
- Tipo de operación. Por ejemplo, en un LABORATORIO (Triglicéridos, glucosa, colesterol…)
- Estado FINAL → Indicando si es correcto o ha habido algún error. (Estado final → Indicando si es correcto o ha habido algún error en el procesamiento (en la validación de los datos) Por ejemplo si hay algún dato que no es lógico.
- Error técnico, si lo hubiera (EXCEPCIONES).

→ **Estos datos (REGISTRO DE PETICIONES) se guardan en un archivo JSON, los cuales se deben cargar al inicio de la segunda aplicación.**

Es importante diferenciar el estado final de la petición de los detalles del error. Por ejemplo, una petición puede terminar con estado «Error de validación» y mostrar el campo concreto que no cumple las reglas.

2. Acerca de / Autores

- Nombre de la aplicación.
- Integrantes de la pareja.
- Fecha de creación.
- Versión del programa.



EJEMPLO JSON

ENVIO DE DATOS

```
{
  "idSolicitud": "SOL-2026-001",
  "muestra": {
    "idMuestra": "M-001",
    "tipo": "Sangre",
    "valida": true
  },
  "pruebas": [
    {
      "codigo": "GLU",
      "valorMedido": 98,
      "unidad": "mg/dL"
    },
    {
      "codigo": "COL",
      "valorMedido": 185,
      "unidad": "mg/dL"
    },
    {
      "codigo": "TRI",
```

```
    "valorMedido": 120,
    "unidad": "mg/dL"
   }
  ]
}
```


DATOS RECIBIDOS


```
{
  "idSolicitud": "SOL-2026-001",
  "estado": "Procesada correctamente",
  "resultados": [
   {
    "codigo": "GLU",
    "valorMedido": 98,
    "unidad": "mg/dL",
    "clasificacion": "Dentro del intervalo"
   },
   {
    "codigo": "COL",
    "valorMedido": 185,
    "unidad": "mg/dL",
    "clasificacion": "Dentro del intervalo"
   },
   {
    "codigo": "TRI",
    "valorMedido": 120,
    "unidad": "mg/dL",
    "clasificacion": "Dentro del intervalo"
   }
}
```